

11 · Window (Analytic) Functions

Why this matters in interviews. "Rank employees within each department", "second highest salary", "running total", "RANK vs DENSE_RANK vs ROW_NUMBER" — window functions are the modern answer to a huge class of interview problems, and the single topic that most clearly separates intermediate from beginner SQL.

In this chapter: the idea of a window · OVER , PARTITION BY , ORDER BY · ROW_NUMBER , RANK , DENSE_RANK , NTILE · running totals and frames · classic patterns (top-N per group, N-th highest, percent of total) · restrictions.

11.1 The idea

A normal aggregate (SUM , AVG , COUNT) **collapses** many rows into one. A **window function** does a similar calculation across a set of related rows — **but keeps every row** in the output, attaching the result as an extra column.

	GROUP BY aggregate	Window function
Output rows	one per group	one per input row (nothing collapses)
You can still show	only grouped columns + aggregates	any column alongside the calculation
Written with	GROUP BY	OVER (...)

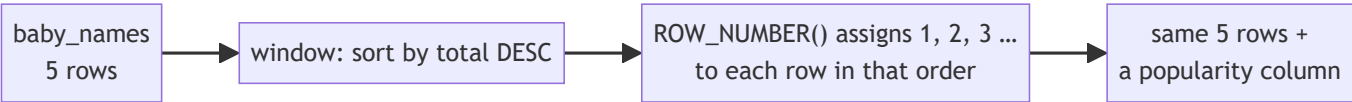
The course video shows the simplest case — *number the rows by popularity*:

```
SELECT gender, name, total,
       ROW_NUMBER() OVER (ORDER BY total DESC) AS popularity
FROM   baby_names;
```

Result

gender	name	total	popularity
Boy	Noah	120	1
Boy	Ethan	115	2
Girl	Emma	106	3
Boy	Jacob	101	4
Girl	Olivia	99	5

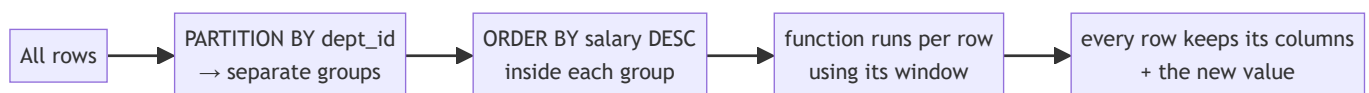
In that statement, **ROW_NUMBER()** is the **function** (what to compute) and **OVER (ORDER BY total DESC)** is the **window** (which rows it looks at, and in what order). Every original row is still there; a new **popularity** column has been added.



11.2 Anatomy

```
function_name ( arguments ) OVER (
  [ PARTITION BY column, ... ]      -- split the rows into independent groups
  [ ORDER BY column [ASC|DESC] ]    -- order rows inside each group
  [ ROWS | RANGE frame ]           -- which rows around the current one to include
)
```

Part	Meaning	If omitted
function	ROW_NUMBER , RANK , DENSE_RANK , NTILE , or an aggregate SUM/AVG/COUNT/MIN/MAX	–
OVER	defines the window — the set of rows the function works on	it is not a window function
PARTITION BY	divides the result set into groups, like GROUP BY but without collapsing rows	one partition = the whole table
ORDER BY (inside OVER)	order of rows inside each partition — needed for ranking and running totals	no ordering, all rows are peers
frame	the range of rows around the current row	see §11.6



Window functions are available in Oracle (8i+), PostgreSQL, SQL Server (2005+) and MySQL **8.0+** (not in MySQL 5.7).

11.3 OVER — an aggregate as a window function

Sample table `staff` (two departments, deliberate salary ties):

staff

id	dept_id	salary	hire_date
1	10	10000	2020-01-01
2	10	9000	2020-06-01
3	10	9000	2021-01-01
4	10	8000	2022-01-01
5	20	7000	2019-05-01
6	20	7000	2020-05-01
7	20	6000	2021-05-01

OVER () — the whole table as one window

```
SELECT id, dept_id, salary,  
       SUM(salary) OVER () AS total_salary  
FROM   staff;
```

Result

id	dept_id	salary	total_salary
1	10	10000	56000
2	10	9000	56000
3	10	9000	56000
4	10	8000	56000
5	20	7000	56000
6	20	7000	56000
7	20	6000	56000

SUM(salary) OVER () computes the total over the **entire table** and repeats it on every row.

PARTITION BY — one window per department

PARTITION BY inside OVER() divides the result set into groups (like GROUP BY) **without collapsing rows**. Each partition acts as a "window" for the function.

```
SELECT id, dept_id, salary,  
       SUM(salary) OVER (PARTITION BY dept_id) AS dept_total_salary  
FROM   staff;
```

Result

id	dept_id	salary	dept_total_salary
1	10	10000	36000
2	10	9000	36000
3	10	9000	36000
4	10	8000	36000
5	20	7000	20000
6	20	7000	20000
7	20	6000	20000

Each department forms a partition (10 → 36,000; 20 → 20,000). SUM(salary) is calculated **per department**, but — unlike GROUP BY — **all employee rows are still shown**.

GROUP BY VS PARTITION BY — same total, different shape

```
SELECT dept_id, SUM(salary) AS dept_total_salary
FROM   staff
GROUP BY dept_id;
```

Result

dept_id	dept_total_salary
10	36000
20	20000

- **GROUP BY** collapses rows into **one row per group**.
- **PARTITION BY** groups data *logically for the window function* but **keeps individual rows**.
- **OVER** defines the scope: the entire table, a partition, or an ordered subset.

A useful share-of-total

```
SELECT id, dept_id, salary,
       ROUND(100 * salary / SUM(salary) OVER (PARTITION BY dept_id), 1) AS pct_of_dept
FROM   staff
ORDER BY dept_id, id;
```

Result

id	dept_id	salary	pct_of_dept
1	10	10000	27.8
2	10	9000	25
3	10	9000	25
4	10	8000	22.2
5	20	7000	35
6	20	7000	35
7	20	6000	30

Impossible with plain **GROUP BY** without joining the totals back in.

11.4 Ranking functions

The **ranking functions** number the rows of each partition according to the window's **ORDER BY**.

Function	Behaviour on ties	Numbering
ROW_NUMBER()	ties are broken arbitrarily — every row gets a unique number	1, 2, 3, 4
RANK()	ties get the same rank ; the next rank is skipped	1, 2, 2, 4

Function	Behaviour on ties	Numbering
DENSE_RANK()	ties get the same rank; no numbers are skipped	1, 2, 2, 3
NTILE(n)	splits the ordered rows into <i>n</i> roughly equal buckets	1,1,2,2 ...

Salaries in department 10: 10000, 9000, 9000, 8000.

```
SELECT dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_number,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rank
FROM   staff
WHERE  dept_id = 10;
```

Result

dept_id	salary	row_number	rank	dense_rank
10	10000	1	1	1
10	9000	2	2	2
10	9000	3	2	2
10	8000	4	4	3

Salary	RANK()	DENSE_RANK()
10000	1	1
9000	2	2
9000	2 (tie → same rank)	2
8000	4 (rank 3 is skipped!)	3 (no skipping)

The rule of thumb: **RANK skips numbers after a tie**; **DENSE_RANK does not**; **ROW_NUMBER never ties**. Because two employees tied at rank 2, the next available **RANK** is 4.

```
SELECT id, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rank_in_dept
FROM   staff
ORDER BY dept_id, rank_in_dept, id;
```

Result

id	dept_id	salary	rank_in_dept
1	10	10000	1
2	10	9000	2
3	10	9000	2
4	10	8000	4
5	20	7000	1

id	dept_id	salary	rank_in_dept
6	20	7000	1
7	20	6000	3

`PARTITION BY dept_id` restarts the numbering for each department; without it the ranking would run across the whole table.

`NTILE(n)` — split into buckets (e.g. top half / bottom half):

```
SELECT id, salary, NTILE(2) OVER (ORDER BY salary DESC) AS half
FROM   staff
ORDER BY salary DESC, id;
```

Result

id	salary	half
1	10000	1
2	9000	1
3	9000	1
4	8000	1
5	7000	2
6	7000	2
7	6000	2

Ranking grouped results

Window functions run **after** `GROUP BY` and `HAVING`, so they can rank an aggregate — *rank the departments by their head-count*. With head-counts 10, 8, 8, 6:

```
CREATE TABLE dept_counts (dept_name VARCHAR2(20), n NUMBER);
INSERT INTO dept_counts VALUES ('Sales', 10);
INSERT INTO dept_counts VALUES ('IT', 8);
INSERT INTO dept_counts VALUES ('HR', 8);
INSERT INTO dept_counts VALUES ('Ops', 6);

SELECT dept_name, n AS headcount,
       RANK()      OVER (ORDER BY n DESC) AS normal_rank,
       DENSE_RANK() OVER (ORDER BY n DESC) AS dense_rank
FROM   dept_counts
ORDER BY n DESC, dept_name;
```

Result

dept_name	headcount	normal_rank	dense_rank
Sales	10	1	1
HR	8	2	2

dept_name	headcount	normal_rank	dense_rank
IT	8	2	2
Ops	6	4	3

Counts [10, 8, 8, 6] → RANK = [1, 2, 2, 4], DENSE_RANK = [1, 2, 2, 3]. The same idea straight from a GROUP BY query:

```
SELECT d.dept_name,
       COUNT(e.emp_id) AS number_of_employees,
       RANK() OVER (ORDER BY COUNT(e.emp_id) DESC) AS normal_rank,
       DENSE_RANK() OVER (ORDER BY COUNT(e.emp_id) DESC) AS dense_rank
FROM   employee e
JOIN   department d ON e.dept_number = d.dept_id
GROUP BY d.dept_name
ORDER BY number_of_employees DESC;
```

Read RANK() OVER (ORDER BY COUNT(e.emp_id) DESC) in two steps: first the rows are **grouped by department**; then the window orders those grouped rows by their count, highest first.

11.5 Running totals — ORDER BY inside OVER

Adding ORDER BY to the window turns an aggregate into a **cumulative** (running) calculation:

```
SELECT id, dept_id, hire_date, salary,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS running_total
FROM   staff
ORDER BY dept_id, hire_date;
```

Result

id	dept_id	hire_date	salary	running_total
1	10	2020-01-01	10000	10000
2	10	2020-06-01	9000	19000
3	10	2021-01-01	9000	28000
4	10	2022-01-01	8000	36000
5	20	2019-05-01	7000	7000
6	20	2020-05-01	7000	14000
7	20	2021-05-01	6000	20000

The cumulative sum of salaries in each department, ordered by hire date. Data is partitioned **by department**; within each department rows are **ordered by hire date**; SUM() becomes a **running total**. Note it restarts for department 20.

Trap — ties in the ORDER BY column. The default frame is `RANGE`, so **rows with equal ORDER BY values are peers and share the same running total**. Order the running total by `salary` (which has ties) and see:

```
SELECT dept_id, salary,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY salary)
AS running_total_default,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY salary ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS running_total_rows
FROM   staff
WHERE  dept_id = 10
ORDER BY salary, id;
```

Result

dept_id	salary	running_total_default	running_total_rows
10	8000	8000	8000
10	9000	26000	17000
10	9000	26000	26000
10	10000	36000	36000

Both 9,000 rows show **26,000** in the first column (8,000 + 9,000 + 9,000 — the tie is treated as one step), but the honest row-by-row total (`ROWS`) is 17,000 then 26,000. To get a strictly row-by-row running total, either add a unique tiebreaker to the `ORDER BY` or use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

11.6 Frames — which rows around the current one?

The **frame** narrows the window *within* a partition relative to the current row.

Frame	Meaning
<code>ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW</code>	from the first row of the partition to the current one (running total)
<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>	the current row and the two before it (a 3-row moving window)
<code>ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING</code>	the whole partition
<code>RANGE ...</code>	like <code>ROWS</code> but by value — ties share a frame

Defaults: no `ORDER BY` ⇒ the frame is the **whole partition**; with `ORDER BY` ⇒ `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`.

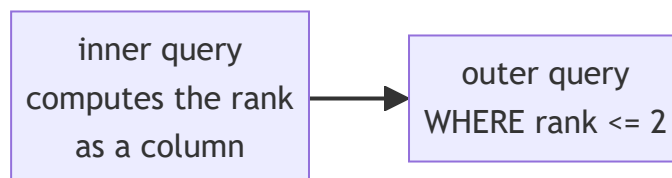
```
SELECT id, hire_date, salary,
       ROUND(AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS
moving_avg_3
FROM   staff
ORDER BY hire_date, id;
```

Result

id	hire_date	salary	moving_avg_3
5	2019-05-01	7000	7000
1	2020-01-01	10000	8500
6	2020-05-01	7000	8000
2	2020-06-01	9000	8667
3	2021-01-01	9000	8333
7	2021-05-01	6000	8000
4	2022-01-01	8000	7667

11.7 Classic patterns

Window functions are computed **after** `WHERE`, `GROUP BY` and `HAVING`, so you **cannot filter on one in the** `WHERE` **clause**. To keep "rank 1" rows, compute the rank in a **subquery** (or a `WITH` clause) and filter outside.



Top-N per group

The two highest-paid employees in each department — on the company table, where department 20 has a tie at 50,000 (Chitra and Dinesh):

```
SELECT dept_id, emp_name, salary, rnk
FROM (SELECT dept_id, emp_name, salary,
            DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
      FROM emp
     WHERE dept_id IS NOT NULL)
WHERE rnk <= 2
ORDER BY dept_id, rnk, emp_name;
```

Result

dept_id	emp_name	salary	rnk
10	Asha	90000	1
20	Bala	70000	1
20	Chitra	50000	2
20	Dinesh	50000	2
30	Farhan	65000	1
30	Gita	35000	2
30	Hari	35000	2

Department 20 (70000, 50000, 50000, 40000, 20000) and department 30 (65000, 35000, 35000) both contain ties, so "top 2" returns **three** people in each. Which ranking function you choose decides how ties behave:

Function in the pattern (WHERE ... <= 2)	Department 20 result	Behaviour
ROW_NUMBER()	Bala, one arbitrary of Chitra/Dinesh	ties are silently cut — usually a bug
RANK()	Bala, Chitra, Dinesh (ranks 1, 2, 2)	ties kept, but see below
DENSE_RANK()	Bala, Chitra, Dinesh (ranks 1, 2, 2)	ties kept and rank numbers are contiguous

RANK and DENSE_RANK only part ways when the tie is at the **top**: for salaries 100, 100, 90, RANK gives 1, 1, **3** — so RANK() <= 2 returns just the two 100s and *misses* the 90, while DENSE_RANK gives 1, 1, **2** and returns all three. "The top N distinct salaries" is a DENSE_RANK question.

The N-th highest salary

DENSE_RANK gives a clean answer even with duplicates (unlike LIMIT / ROWNUM). *Second highest salary in the company:*

```
SELECT DISTINCT salary AS second_highest_salary
FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM emp)
WHERE rnk = 2;
```

Result

second_highest_salary
70000

Change = 2 to = N for the N-th. Add PARTITION BY dept_id for the N-th **per department**.

Highest-paid in each department (with all ties)

```
SELECT dept_id, emp_name, salary
FROM (SELECT dept_id, emp_name, salary,
            RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
      FROM emp WHERE dept_id IS NOT NULL)
WHERE rnk = 1
ORDER BY dept_id;
```

Result

dept_id	emp_name	salary
10	Asha	90000
20	Bala	70000
30	Farhan	65000

Remove duplicates — keep one row per group

`ROW_NUMBER()` OVER (PARTITION BY <what makes a duplicate> ORDER BY <which to keep>) numbers the copies; keep number 1 (or delete the rest):

```
SELECT id, dept_id, salary
FROM (SELECT id, dept_id, salary,
            ROW_NUMBER() OVER (PARTITION BY dept_id, salary ORDER BY id) AS copy_no
      FROM staff)
WHERE copy_no = 1
ORDER BY dept_id, salary DESC;
```

Result

id	dept_id	salary
1	10	10000
2	10	9000
4	10	8000
5	20	7000
7	20	6000

Rows with the same (department, salary) collapse to the first (lowest `id`).

The department average next to each employee

An alternative to the correlated subquery of Chapter 08:

```
SELECT emp_name, dept_id, salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id)) AS dept_avg,
       salary - ROUND(AVG(salary) OVER (PARTITION BY dept_id)) AS diff_from_avg
FROM emp
WHERE dept_id = 30
ORDER BY emp_id;
```

Result

emp_name	dept_id	salary	dept_avg	diff_from_avg
Farhan	30	65000	45000	20000
Gita	30	35000	45000	-10000

emp_name	dept_id	salary	dept_avg	diff_from_avg
Hari	30	35000	45000	-10000

11.8 Rules and restrictions

- Window functions are allowed **only in the SELECT list and ORDER BY** — not in WHERE, GROUP BY or HAVING. Filter through a subquery or CTE.
- They are evaluated **after** GROUP BY / HAVING, so they can rank or sum *aggregated* values, and **before** the final ORDER BY / FETCH.
- Ranking functions (ROW_NUMBER, RANK, DENSE_RANK, NTILE) **need an ORDER BY** inside OVER; without it the numbering is meaningless.
- PARTITION BY may list several columns; NULLs in a partition column form one partition.
- Ties: ROW_NUMBER is only deterministic if the ORDER BY is unique — add a tiebreaker (ORDER BY salary DESC, emp_id).

Interview corner

Q. What is a window function? How does it differ from GROUP BY? It computes over a set of related rows **without collapsing them**: every input row stays, with the result added as a column. GROUP BY returns one row per group.

Q. RANK vs DENSE_RANK vs ROW_NUMBER? For values 100, 90, 90, 80: ROW_NUMBER 1,2,3,4 · RANK 1,2,2,4 · DENSE_RANK 1,2,2,3.

Q. What do PARTITION BY and OVER do? OVER defines the window; PARTITION BY splits rows into independent groups inside it, restarting the calculation per group.

Q. Find the second-highest salary. SELECT DISTINCT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM emp) WHERE r = 2; — DENSE_RANK because duplicates must not use up ranks.

Q. Top 3 earners per department? DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) in a subquery, then WHERE rnk <= 3.

Q. Why can't I write WHERE RANK() OVER (...) = 1? Window functions are evaluated after WHERE; wrap the query so the rank is a column, then filter outside.

Q. What is a running total and how do you compute it? SUM(x) OVER (ORDER BY d) (add PARTITION BY for per-group). Beware ties in the ORDER BY column — default RANGE frames treat tied rows as one step.

Q. Can you use an aggregate with OVER? Yes — SUM, AVG, COUNT, MIN, MAX all become window functions with OVER.

Q. How do you delete duplicates keeping one row? Number copies with ROW_NUMBER() OVER (PARTITION BY dup_cols ORDER BY id) and delete where the number > 1 (DELETE ... WHERE ROWID IN (...) in Oracle).

Quick recap

- A window function calculates across related rows **and keeps every row**: `function() OVER (PARTITION BY ... ORDER BY ... [frame])`.
- `OVER ()` = whole table · `PARTITION BY` = per-group (no collapsing) · `ORDER BY` inside `OVER` = ranking or **running total**.
- `ROW_NUMBER` unique · `RANK` ties, then a gap · `DENSE_RANK` ties, no gap · `NTILE(n)` buckets.
- Default frame with `ORDER BY` is `RANGE ... CURRENT ROW` — ties share a running total; use `ROWS` for row-by-row.
- Can't be used in `WHERE` : compute in a subquery, filter outside. Recipes: **top-N per group**, **N-th highest** (`DENSE_RANK`), de-duplication (`ROW_NUMBER = 1`), share-of-total.